

بسمی تعالی



دانشگاه شهید باهنر کرمان

پروژه نظریه زبان ها و ماشین ها

نام استاد: خانم دکتر شیما شفیعی

عنوان پروژه:

شبیه ساز NFA با انتقال ϵ و حذف ϵ برای تبدیل به طول NFA بدون ϵ

نام و نام خانوادگی اعضای گروه:

علیرضا ریانی (بخش پیاده سازی)

دانیال دهقانی (بخش پروپوزال)

علی خواجویی (بخش ویراست ، تحقیق و جمع آوری اطلاعات)

علی دسترنج (بخش پیاده سازی)

نیمسال تحصیلی دوم ۱۴۰۵-۱۴۰۴

فهرست مطالب

1_مقدمه	3
2_بیان مسئله	4
2.1_اتوماتای متناهی چیست	4
2.2_تفاوت NFA ، DFA و $\varepsilon-NFA$	4
2.3_چرا باید $\varepsilon-NFA$ به NFA تبدیل شود	4
2.4_کاربرد این تبدیل در نظریه زبان ها و ماشین ها	5
3_اهداف پروژه	6
4_الگوریتم تبدیل $\lambda-NFA$ به NFA	7
4.1_روند محاسبه $\lambda - closure$	7
4.2_مراحل تبدیل	7
5_مثال کاربردی	8
5.1_اتوماتای اولیه ($\lambda-NFA$)	8
5.2_محاسبات $\lambda-closure$	8
5.3_جدول انتقال (NFA)	8
5.4_نمودار نهایی (NFA)	9
6_طراحی و پیاده سازی	10
7_آزمایش و نتایج	11
8_نتیجه گیری	14
9_منابع و لینک ها	15

1_مقدمه

در درس نظریه زبان ها و ماشین ها، اتوماتاهای متناهی یکی از مهم ترین ابزارها برای مدل سازی و تحلیل زبان های منظم هستند. این اتوماتاها در انواع مختلفی مانند DFA ، NFA و $\epsilon-NFA$ تعریف می شوند. هر یک از این مدل ها ویژگی ها و کاربردهای خاص خود را دارند، اما از نظر قدرت بیان معادل یکدیگر هستند.

در برخی از اتوماتاها انتقال هایی بدون مصرف ورودی وجود دارد که با نماد ϵ (اپسیلن) نمایش داده می شوند. این نوع انتقال ها باعث ساده تر شدن طراحی اتوماتا می شوند، اما برای تحلیل و پیاده سازی عملی معمولاً لازم است این انتقال ها حذف شده و اتوماتای معادل بدون ϵ ساخته شود.

در این پروژه، ابتدا مفاهیم مربوط به اتوماتا های DFA ، NFA و $\epsilon-NFA$ بررسی می شود و سپس روش حذف انتقال های ϵ و تبدیل $\epsilon-NFA$ به NFA توضیح داده خواهد شد. در ادامه، این الگوریتم با استفاده از یک مثال عملی بررسی شده و در نهایت پیاده سازی آن با استفاده از پایتون و جاوا اسکریپت ارائه می شود.

2_ بیان مسئله

2.1_ اتوماتای متناهی چیست

اتوماتای متناهی یکی از مدل‌های محاسباتی در نظریه زبان‌ها و ماشین‌ها است که برای تشخیص و پردازش زبان‌های منظم به کار می‌رود. این مدل از مجموعه‌ای از حالت‌ها، یک حالت شروع، یک یا چند حالت پذیرش و مجموعه‌ای از انتقال‌ها تشکیل شده است. اتوماتای متناهی با دریافت رشته‌ای از ورودی‌ها، بر اساس قوانین انتقال از حالتی به حالت دیگر حرکت می‌کند و در نهایت مشخص می‌کند که رشته ورودی توسط زبان مورد نظر پذیرفته می‌شود یا خیر. به دلیل سادگی ساختار و قدرت مناسب در مدل‌سازی بسیاری از مسائل، اتوماتاهای متناهی کاربرد گسترده‌ای در علوم کامپیوتر دارند.

2.2_ تفاوت NFA ، DFA و $\epsilon-NFA$

سه نوع رایج از اتوماتاهای متناهی شامل DFA ، NFA و $\epsilon-NFA$ هستند. در اتوماتای متناهی قطعی (DFA)، برای هر حالت و هر نماد ورودی دقیقاً یک انتقال مشخص وجود دارد و ماشین در هر لحظه فقط در یک حالت قرار دارد. در مقابل، در اتوماتای متناهی غیرقطعی (NFA) ممکن است برای یک نماد ورودی چندین انتقال از یک حالت وجود داشته باشد و ماشین بتواند به چند حالت مختلف حرکت کند. نوع سوم، $\epsilon-NFA$ است که علاوه بر ویژگی‌های NFA ، دارای انتقال‌هایی بدون مصرف ورودی است که با نماد ϵ نشان داده می‌شوند. این انتقال‌ها به ماشین اجازه می‌دهند بدون خواندن نماد جدید، از یک حالت به حالت دیگر منتقل شود.

2.3_ چرا باید $\epsilon-NFA$ به NFA تبدیل شود

وجود انتقال‌های ϵ در طراحی اتوماتا می‌تواند فرآیند ساخت ماشین را ساده‌تر کند، اما در برخی موارد باعث پیچیدگی در تحلیل و پیاده‌سازی می‌شود. به همین دلیل معمولاً لازم است اتوماتای $\epsilon-NFA$ به یک NFA معادل بدون انتقال ϵ تبدیل شود. این تبدیل باعث می‌شود ساختار ماشین ساده‌تر شده و بررسی مسیرهای انتقال و پردازش ورودی‌ها

به شکل واضح‌تری انجام گیرد. همچنین بسیاری از الگوریتم‌ها و پیاده‌سازی‌های عملی در نظریه اتوماتا ترجیح می‌دهند با مدل‌هایی کار کنند که فاقد انتقال‌های ϵ باشند.

2.4_ کاربرد این تبدیل در نظریه زبان‌ها و ماشین‌ها

تبدیل ϵ -NFA به NFA یکی از مراحل مهم در تحلیل و طراحی ماشین‌های حالت متناهی است. این تبدیل در فرآیندهای مختلفی مانند طراحی کامپایلرها، تحلیل عبارات منظم و ساخت سیستم‌های تشخیص الگو کاربرد دارد. با حذف انتقال‌های ϵ ، ساختار اتوماتا ساده‌تر شده و امکان تبدیل آن به DFA نیز راحت‌تر فراهم می‌شود. در نتیجه این فرآیند نقش مهمی در ساده‌سازی مدل‌های محاسباتی و افزایش کارایی الگوریتم‌های مرتبط با زبان‌های منظم ایفا می‌کند.

3_اهداف پروژه

هدف از این پروژه، پیاده سازی و درک فرآیند تبدیل ماشین های غیر قطعی

($\lambda - NFA$) به ماشین های NFA استاندارد است. اهداف اصلی این مطالعه و پیاده سازی عبارتند از:

- آشنایی با ساختار $\lambda - NFA$: بررسی دقیق اجزای تشکیل دهنده اتوماتای متناهی غیر قطعی با انتقال های ε ، شامل حالت ها، الفبا، توابع انتقال و حالت های شروع و پذیرش.
- درک مفهوم $\lambda - closure$: تحلیل دقیق تابع «بستار اپسیلون» به عنوان قلب محاسباتی تبدیل اتوماتاها و چگونگی استفاده از آن برای پیمایش مسیرهای غیرمستقیم در ماشین.
- یادگیری الگوریتم تبدیل $\lambda - NFA$ به NFA : بررسی گام به گام منطق ریاضی حذف انتقال های ε و ساخت توابع انتقال جدید برای دستیابی به یک NFA معادل.
- پیاده سازی شبیه ساز: طراحی و توسعه یک ابزار نرم افزاری ساده جهت شبیه سازی اتوماتا، به منظور بررسی و تحلیل پذیرش یا عدم پذیرش رشته های ورودی توسط ماشین های طراحی شده.

۴_الگوریتم تبدیل λ -NFA به NFA

برای حذف انتقال های λ و تبدیل یک λ -NFA به NFA معادل، از الگوریتم زیر استفاده می شود.

4.1_روند محاسبه λ -closure

بستار اپسیلون (λ -closure) برای یک حالت q ، مجموعه ای از تمام حالت هایی است که می توان با طی کردن صفر یا تعدادی انتقال λ از حالت q به آن ها رسید.

مراحل محاسبه:

۱. حالت q را در مجموعه λ -closure قرار دهید.
۲. تمام حالت هایی که با یک انتقال λ از q مستقیماً در دسترس هستند را به مجموعه اضافه کنید.
۳. برای تمام حالت های جدیدی که به مجموعه اضافه شده اند، همین کار را تکرار کنید (به صورت بازگشتی) تا زمانی که دیگر حالت جدیدی باقی نماند.

4.2_مراحل تبدیل

برای ساختن NFA معادل از λ -NFA:

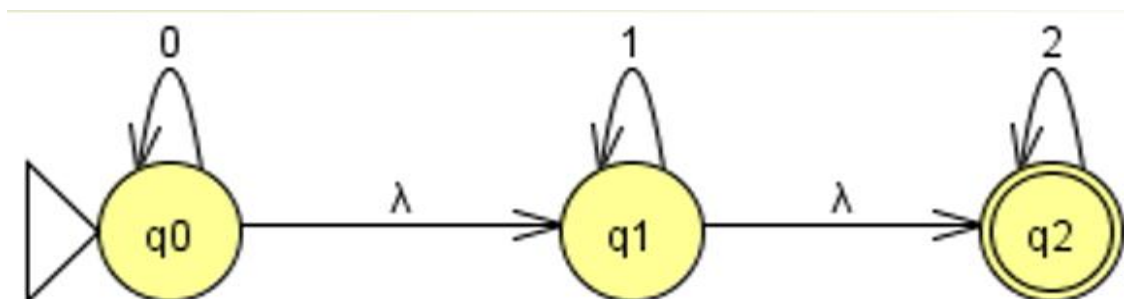
۱. تعیین حالت های NFA: همان حالت های λ -NFA هستند.
۲. تعیین حالت شروع: اگر q_0 حالت شروع λ -NFA باشد، حالت شروع NFA برابر است با λ -closure(q_0).
۳. تعیین حالت های پذیرش: هر حالتی در NFA که حداقل یکی از اعضای آن در λ -NFA جزء «حالت های پذیرش» باشد، به عنوان حالت پذیرش در NFA جدید انتخاب می شود.
۴. ساخت توابع انتقال (δ'): برای هر حالت S در NFA و هر نماد ورودی a (در الفبای Σ):

$$\delta'(S, a) = \lambda\text{-closure}\left(\bigcup_{s \in S} \delta(s, a)\right) \quad \bullet$$

۵. مثال کاربردی

5.1_ اتوماتای اولیه (λ -NFA)

همان‌طور که در تصویر مشاهده می‌شود، اتوماتای λ -NFA دارای ۳ حالت $\{q_0, q_1, q_2\}$ است که انتقال‌های λ بین آن‌ها برقرار است.



5.2_ محاسبات λ -closure

قبل از تبدیل، باید بستار اسیلون برای هر حالت محاسبه شود:

$$\lambda - \text{closure}(q_0) = \{q_0, q_1, q_2\} \quad \bullet$$

$$\lambda - \text{closure}(q_1) = \{q_1, q_2\} \quad \bullet$$

$$\lambda - \text{closure}(q_2) = \{q_2\} \quad \bullet$$

5.3_ جدول انتقال (NFA)

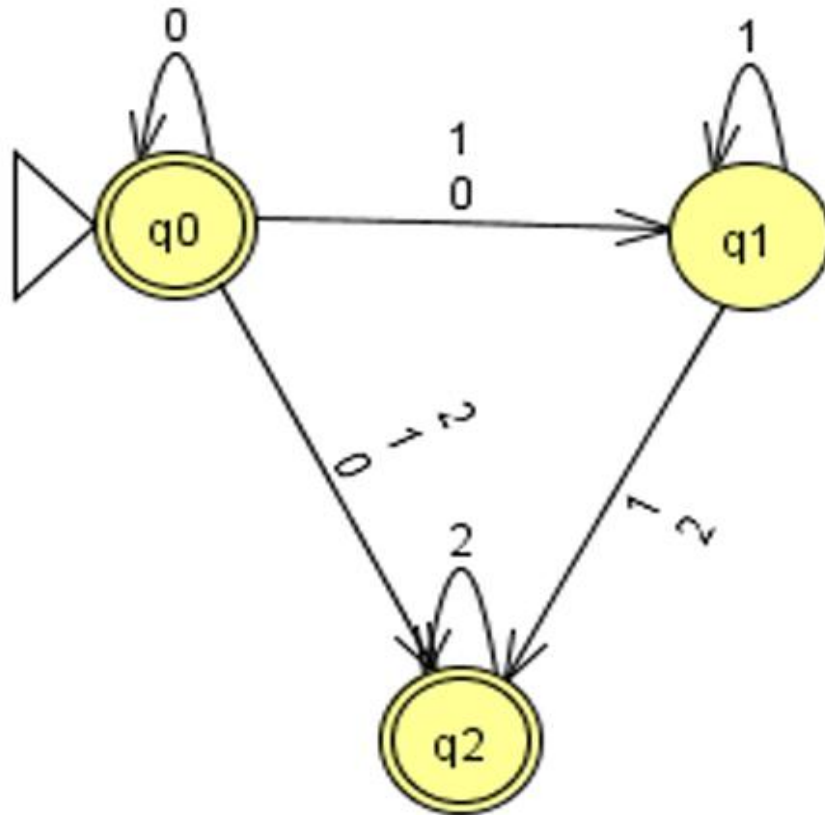
با استفاده از فرمول $\delta'(q, a) = \lambda - \text{closure}(\delta(\lambda - \text{closure}(q), a))$ ، جدول انتقال NFA به دست می‌آید که در زیر نمایش داده شده است:

δ	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\{\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\{\}$	$\{\}$	$\{q_2\}$

5.4_ نمودار نهایی (NFA)

نمودار حاصل از این تبدیل در تصویر زیر (سمت راست) ترسیم

شده است که در آن انتقال‌های λ حذف شده و مسیرها مستقیم شده‌اند.



مشاهده می‌شود که در NFA نهایی، حالت‌های q_0 و q_1 و q_2 همگی حالت‌های پذیرش هستند (چون در محاسبات F' ، بستر اپسیلون آن‌ها با حالت پذیرش اولیه اشتراک دارد). این موضوع نشان‌دهنده حفظ دقیق منطق پذیرش رشته‌ها در حین تبدیل است.

6_طراحی و پیاده سازی

<https://github.com/AlirezaRayyani/lambda-nfa-to-nfa/blob/main/Python/Automata.py>

7_آزمایش و نتایج

۱. ابتدا کاربر NFA مورد نظر را با فرمتی که برنامه میپذیرد را وارد میکند.

۲. سپس برنامه به صورت خودکار مجموعه حالات ، الفبا ، حالت شروع و پایانی را تشخیص میدهد.

از وضعیت	نماد	به وضعیت	عملیات
q0	λ	q1	حذف
q0	λ	q3	حذف
q1	a	q2	حذف
q3	b	q4	حذف
q4	a	q4	حذف

پاک کردن همه افزودن گذار جدید

۳. برنامه تابع گذار را بر اساس ورودی کاربر پیدا میکند و نشان میدهد.

رسمه رد شد

خروجی متنی حذف λ و تبدیل به NFA شبه‌سازی رشته محاسبه λ-Closure

پاک کردن گراف رسم NFA بدون λ رسم NFA-λ

خروجی‌ها

A-Closure شبه‌سازی تبدیل NFA اطلاعات ماشین

شبه‌سازی رشته: I

حالت شروع: q0

حالت‌های نهایی: {q2, q4}

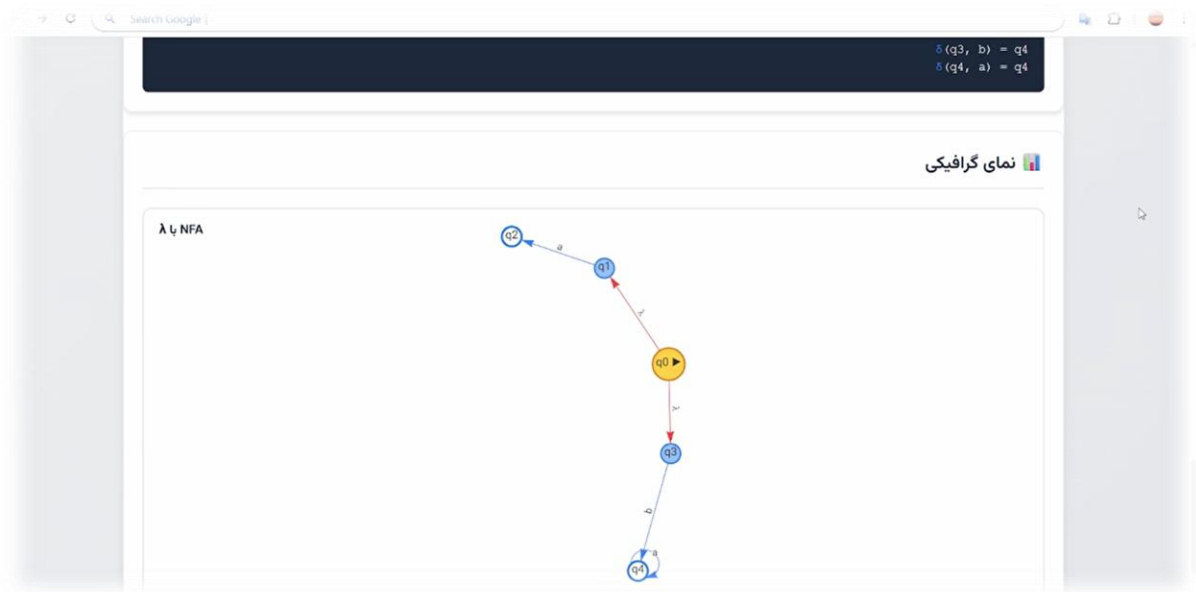
مراحل شبه‌سازی:

مرحله 0: $\lambda\text{-closure}(q_0) = \{q_0, q_1, q_3\}$

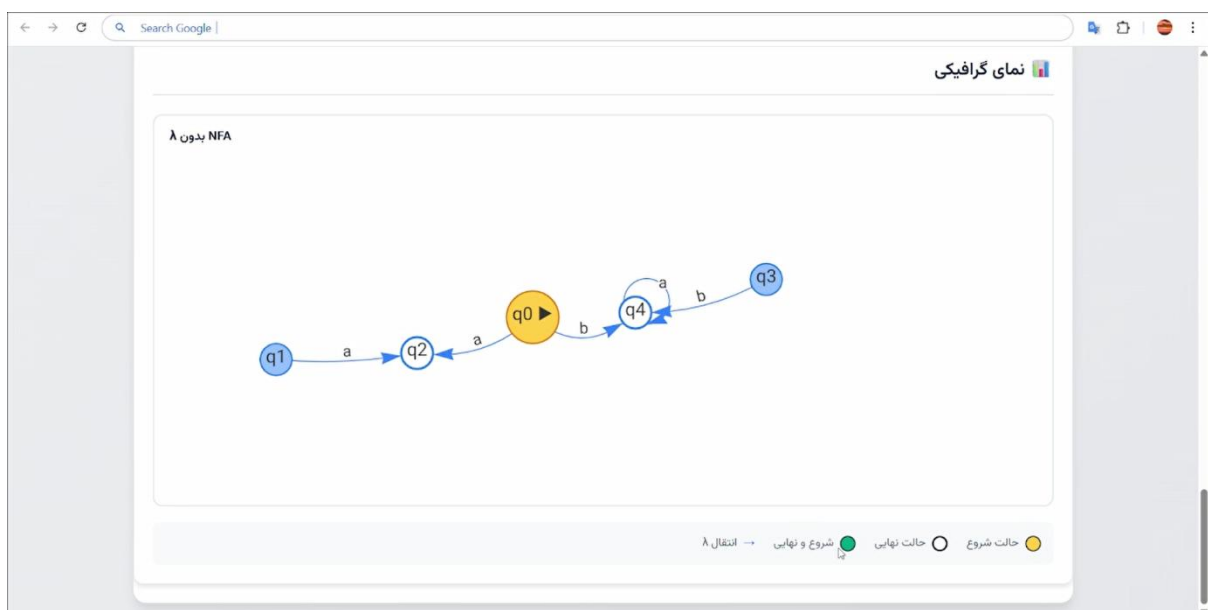
نتیجه نهایی: رد شده

حالت‌های پایانی: {q0, q1, q3}

۴. برنامه یک خروجی متنی به کاربر نشان می دهد که کاربر میتواند بین انواع خروجی ها سوییچ کند.



۵. برنامه نمای گرافیکی را رسم میکند.



۶. سپس جواب را به صورت گرافیکی نشان میدهد.

۸_ نتیجه گیری

در این پروژه، مفهوم اتوماتای متناهی غیر قطعی با انتقال λ و روش تبدیل آن به اتوماتای متناهی غیرقطعی بدون انتقال λ مورد بررسی قرار گرفت. وجود انتقال‌های λ در طراحی اولیه اتوماتا باعث ساده‌تر شدن مدل‌سازی و نمایش برخی زبان‌ها می‌شود، زیرا در این حالت ماشین می‌تواند بدون مصرف هیچ نمادی از ورودی، از یک حالت به حالت دیگر منتقل شود. با این حال، برای تحلیل دقیق‌تر و پیاده‌سازی عملی، حذف این انتقال‌ها ضروری است.

تبدیل λ -NFA به NFA از این جهت اهمیت دارد که ساختار اتوماتا را ساده‌تر و قابل فهم‌تر می‌کند، بدون آنکه زبان پذیرفته شده توسط ماشین تغییر کند. در این فرآیند، با استفاده از مفهوم λ -closure، تمام حالت‌هایی که از طریق انتقال‌های λ قابل دسترسی هستند محاسبه می‌شوند و سپس تابع انتقال جدید بر اساس آن ساخته می‌شود. در نتیجه، اتوماتای نهایی رفتاری معادل با ماشین اولیه خواهد داشت، اما دیگر نیازی به انتقال‌های λ نخواهد بود.

این تبدیل همچنین ارتباط مستقیمی با DFA دارد. در نظریه اتوماتا، برای هر NFA یک DFA معادل وجود دارد که همان زبان را می‌پذیرد. بنابراین، زمانی که یک λ -NFA ابتدا به NFA و سپس در صورت نیاز به DFA تبدیل می‌شود، نشان داده می‌شود که همه این مدل‌ها از نظر قدرت بیان معادل هستند. تفاوت آن‌ها تنها در نحوه نمایش و ساختار اجرایی آن‌هاست. معمولاً DFA برای پیاده‌سازی‌های عملی مناسب‌تر است، زیرا در هر لحظه فقط یک مسیر مشخص را دنبال می‌کند.

از دیدگاه نظریه زبان ها، این موضوع اهمیت زیادی دارد، زیرا نشان می دهد زبان های منظم را می توان با مدل های مختلفی مانند NFA ، DFA و λ -NFA نمایش داد. در واقع، این سه مدل همگی دقیقاً کلاس زبان های منظم را توصیف می کنند. به همین دلیل، تبدیل بین این مدل ها یکی از مباحث پایه ای و اساسی در نظریه زبان ها و ماشین ها محسوب می شود.

در مجموع، این پروژه نشان داد که اگرچه استفاده از انتقال های λ در طراحی اتوماتا می تواند فرآیند ساخت را ساده تر کند، اما برای رسیدن به مدلی استانداردتر، قابل تحلیل تر و مناسب تر برای پیاده سازی، تبدیل آن به NFA و سپس در صورت نیاز به DFA ضروری و مفید است.

9_منابع و لینک ها

۱. <https://www.daneshjooyar.com>-دوره-آموزش-درس-نظریه-زبان-ها-و-ماشین /
۲. An Introduction to Formal Languages and Automata – Vth Edition (Peter Linz) An Introduction to Formal Languages and Automata – Vth Edition (Peter Linz)